TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

24122002 - Đinh Như Phát

24122019 - Từ Văn Khôi

24122031 - Trần Lê Minh Đức

PROJECT 1
FILE & PROCESS MANAGEMENT

Môn học: Hệ điều hành

Lớp: 24TNT

GIÁO VIÊN HƯỚNG DẪN

ThS. Phạm Tuấn Sơn

ThS. Lê Viết Long

Tp. Hồ Chí Minh, ngày 20 tháng 10 năm 2026

1 Phần tìm hiểu

1.1 Quản lý file đang mở

***Trường* `ofile[]` *của* `struct proc`**

`ofile[NOFILE]` trong `struct proc` (`kernel/proc.h`) với `NOFILE = 16` (`kernel/param.h`): bảng file descriptor riêng của mỗi tiến trình, gồm các con trỏ `struct file*`. Chỉ số của mảng chính là fd mà user dùng trong `read(fd,...)`, `write(fd,...)`, `close(fd)`.

***Trường* `file[]` *của* `struct ftable`**

`ftable.file[NFILE]`(`kernel/file.c`) với `NFILE = 100`: bảng file toàn hệ thống dùng chung cho mọi tiến trình, lưu trực tiếp các `struct file`, hệ thống có tối đa 100 file mở cùng lúc.

***Mối quan hệ giữa* `ofile[]` *và* `file[]`**

Mỗi phần tử `ofile[fd]` của một tiến trình lưu một con trỏ trỏ tới đúng một phần tử trong mảng `ftable.file[]`. Nhiều phần từ `ofile[]` của một hay nhiều tiến trình có thể cùng trỏ tới cùng một phần tử `struct file` duy nhất trong `ftable.file[]`. Trường `ref` trong `struct file` đếm số lượng phần tử `ofile[]` đang trỏ tới nó.

***Luồng thực thi của system call* `open`**

1. Từ user vào kernel. Chương trình gọi `open(path, omode)`; stub nạp `SYS_open = 15` vào `a7` rồi thực hiện `ecall`; `ecall` gây trap, CPU nhảy vào `usertrap()`; hàm này nhận diện syscall và gọi `syscall()`, tra bảng con trỏ hàm `syscalls[]` để gọi đúng `sys_open()`.
2. Xử lý trong `sys_open()`. Hàm gồm các bước nhỏ:
   - Lấy tham số từ trapframe: gọi `argint` và `argstr` để đọc từ `p->trapframe->a0/a1`.
   - Tìm hoặc tạo inode: nếu `omode` có cờ `O_CREATE` thì gọi `create`, ngược lại gọi `namei`.
     - `create()`: tạo file thường mới tại `path`. Hàm tìm thư mục cha, cấp inode mới bằng `ialloc()`, rồi thêm mục tên file vào thư mục cha bằng `dirlink()`.
     - `namei()`: chỉ tìm inode của file đã tồn tại. Hàm duyệt `path` từ thư mục gốc (hoặc thư mục hiện tại), tìm từng thành phần tên bằng `dirlookup()`, và trả về inode của thành phần cuối, hoặc `NULL` nếu không tìm thấy.
   - Xin một `struct file` toàn cục: gọi `filealloc()`, hàm này quét `ftable.file[NFILE]` để tìm ô có `ref==0`.
   - Xin một file descriptor trong tiến trình: gọi `fdalloc(f)`, hàm này quét `p->ofile[NOFILE]` tìm ô trống nhỏ nhất rồi gán con trỏ `f` vào đó.
   - Diền thông tin vào `struct file`: ghi loại file (thiết bị hoặc inode thường), gán `f->ip` trỏ tới inode, và đặt quyền đọc/ghi theo các cờ trong `omode`. Trả về `fd`.
3. Quay lại user space. `syscall()` lưu fd vào `p->trapframe->a0`; `usertrapret()` khôi phục trapframe và nhảy về ngay sau `ecall` trong stub `open`, `open()` trả về fd.

***So sánh file descriptor:* `open()`, `dup()` *và* `fork()`**

Cả ba đều tạo thêm fd trỏ tới một file, nhưng khác nhau ở việc có tạo `struct file` mới và có chia sẻ offset hay không (Bảng 1).

- `open()`: mỗi lần gọi tạo ra một `struct file` mới với offset riêng.
- `dup()`: không tạo `struct file` mới mà chỉ thêm một fd trỏ vào `struct file` sẵn có và tăng `ref`. Vì offset nằm trong `struct file` nên hai fd này chỉ có một offset duy nhất.
- `fork()`: tiến trình con sao chép bảng `ofile[]` của cha, với mỗi fd đang mở thì gọi `filedup()`. Con giữ nguyên số fd như cha và dùng chung `struct file` nên dùng chung offset.

Tiêu chí	`open()`	`dup()`	`fork()`
Cấp `struct file` mới trong `ftable`?	Có, gọi `filealloc()`	Không	Không
Cấp fd mới?	Có, `fdalloc()` chọn ô trống nhỏ nhất trong `ofile[]`	Có, `fdalloc()` chọn ô trống nhỏ nhất	Không tìm ô trống: tiến trình con có mảng `ofile[]` riêng, cùng số fd với cha
Giá trị `ref` của `struct file`	Đặt bằng 1	Tăng thêm 1 (`filedup()`)	Tăng thêm 1 cho mỗi fd đang mở của cha (`filedup()`)
Offset	Riêng cho mỗi lần `open()`, bắt đầu từ 0	Dùng chung với fd cũ	Dùng chung giữa cha và con
Quyền đọc/ghi	Theo cờ `omode` của lần mở này	Giống fd cũ	Giống fd của cha
Inode	Tìm hoặc tạo bằng `namei()`/`create()`	Cùng inode với fd cũ	Cùng inode với fd của cha

Bảng 1: So sánh `open()`, `dup()` và `fork()`

1.2 Quản lý tiến trình

***Các trường* `pid`, `state`, `context` *của* `struct proc`**

- `pid`: số định danh duy nhất của tiến trình. Giá trị được cấp bởi `allocproc()` thông qua `allocpid()`, hàm này lấy `pid_lock` rồi tăng biến toàn cục `nextpid`.
- `state`: trạng thái hiện tại của tiến trình, kiểu `enum procstate`. Trường này được bảo vệ bởi `p->lock`. Các giá trị và ý nghĩa được tóm tắt trong Bảng 2.
- `context`: kiểu `struct context`, lưu các thanh ghi callee-saved (`ra`, `sp`, `s0`–`s11`) của luồng kernel khi tiến trình bị chuyển ra khỏi CPU.

Trạng thái	Ý nghĩa
`UNUSED`	Ô trong `proc[]` còn trống
`USED`	Vừa được cấp, chưa sẵn sàng chạy
`RUNNABLE`	Sẵn sàng chạy, chờ `scheduler()` chọn
`RUNNING`	Đang chạy trên một CPU
`SLEEPING`	Đang ngủ, chờ một sự kiện
`ZOMBIE`	Đã kết thúc, chờ cha gọi `wait()`

Bảng 2: Các giá trị của `state`

***Luồng thực thi của system call* `fork`**

1. Từ user đến kernel. `fork()` trả về `pid` của con cho cha, 0 cho con, và -1 nếu thất bại. Stub nạp `SYS_fork = 1` vào `a7`, `ecall`, qua `usertrap()` và `syscall()` tới `sys_fork()`, hàm này gọi thẳng `fork()` trong `kernel/proc.c`.
2. Xử lý trong `fork()`. Hàm gồm các bước nhỏ:
   - Cấp phát một process con: gọi `allocproc()` tìm slot `UNUSED` trong `proc[NPROC]`, cấp `pid`, `trapframe`, `pagetable` rỗng và đặt `context.ra = forkret`.
   - Copy bộ nhớ user từ cha sang con :gọi `uvmcopy()` cấp trang vật lý mới và sao chép bộ nhớ user của cha, nên không gian địa chỉ của con độc lập với cha.
   - Sao chép `*(np->trapframe) = *(p->trapframe)` để con tiếp tục từ đúng vị trí của cha, rồi đặt `np->trapframe->a0 = 0`. Dây là cơ chế khiến con nhận 0 từ `fork()`.
   - Kế thừa tài nguyên: với mỗi fd đang mở của cha gọi `filedup()` (cha và con dùng chung `struct file`); `idup(p->cwd)` để con kế thừa thư mục hiện tại.
   - Lưu `np->pid` vào biến local rồi `release(&np->lock)`, vì `pid` chỉ được đọc khi đang giữ

p->lock.

- Thiết lập quan hệ cha–con: dưới wait_lock, gán np->parent = p để wait() của cha tìm thấy con và exit()/reparent() biết ai là cha.
- Đưa con vào hàng chờ lập lịch: đặt np->state = RUNNABLE (dưới np->lock) để scheduler() có thể chọn con chạy.
- Trả về cho cha: return pid;. Giá trị này được sys_fork() chuyển tiếp, và syscall() ghi vào p->trapframe->a0 của cha.

Nếu allocproc() hoặc uvmcopy() thất bại, fork() giải phóng phần đã cấp bằng freeproc() và trả về −1.

3. Hai nhánh quay về user space.
   - Ở process cha: syscall() ghi pid vào a0; usertrapret() quay về sau ecall và fork() trả về pid.
   - Ở process con: chưa từng chạy trong kernel nên không có vị trí cũ để quay lại. Khi scheduler() chọn con, swtch() nhảy vào forkret() (do context.ra = forkret); hàm này nhả p->lock còn giữ từ scheduler, rồi usertrapret() khôi phục trapframe với a0 = 0. Hàm fork() ở process con trả về 0.

Cơ chế lập lịch: **scheduler(), sched(), yield()**

Ba hàm này cùng nhau tạo thành vòng lặp lập lịch của xv6. Mỗi lõi CPU chạy một thể hiện riêng của scheduler(), còn sched() và yield() là hai cổng vào mà một process đang chạy dùng để nhường CPU lại cho scheduler().

scheduler() Hàm này được gọi một lần duy nhất bởi mỗi CPU khi khởi động xong, và không bao giờ trả về. Nó chạy một vòng lặp vô hạn:

- Gọi intr_on() để bật lại ngắt, tránh deadlock trong trường hợp mọi process đều đang chờ mà không có ngắt timer nào xảy ra để đánh thức chúng.
- Quét tuần tự bảng proc[NPROC]. Với mỗi p, gọi acquire(&p->lock) trước khi kiểm tra state. Nếu state==RUNNABLE, đổi thành RUNNING, gán c->proc = p, rồi gọi swtch(&c ->context, &p->context).
- Lời gọi swtch() lưu ngữ cảnh của CPU hiện tại vào c->context và nạp ngữ cảnh của process được chọn từ p->context, khiến CPU bắt đầu chạy process đó. Hàm scheduler() tạm thời bị treo lại tại đúng điểm gọi swtch().
- Khi process này nhường CPU (thông qua sched()), swtch() trả quyền điều khiển về lại đúng điểm này trong scheduler(). Lúc đó, c->proc được đặt lại thành 0 gọi release(&p->lock), rồi vòng for tiếp tục quét sang proc kế tiếp.
- Nếu một vòng quét hết toàn bộ bảng mà không tìm thấy proc nào RUNNABLE (found == 0), CPU thực hiện lệnh wfi để tạm nghỉ, tránh lãng phí chu kỳ CPU khi không có việc gì để chạy.

sched() Hàm này được gọi bởi một process đang chạy khi nó muốn nhường CPU lại cho scheduler(). Trước khi swtch() thật sự xảy ra, sched() kiểm tra một loạt điều kiện bằng panic nếu sai:

- p->lock phải đang được giữ bởi process gọi, vì sau đó state sẽ được scheduler đọc mà không cần acquire lại.
- Số lượng khóa đang giữ (mycpu()->noff) phải đúng bằng 1, tức là p->lock là khóa duy nhất còn giữ tại thời điểm chuyển ngữ cảnh.
- p->state không được là RUNNING, vì process gọi sched() phải đã tự đổi trước đó.
- Ngắt phải đang bị tắt (intr_get() trả về 0), để tránh một ngắt xảy ra giữa lúc chuyển

ngữ cảnh.

Sau khi qua các kiểm tra, `sched()` lưu lại `intena` (trạng thái ngắt trước khi `push_off()`), gọi `swtch(&p->context, &mycpu()->context)` để chuyển quyền điều khiển về đúng điểm trong `scheduler()` đang chờ, và khi CPU này quay lại chạy process này lần sau, khôi phục lại `mycpu()->intena`.

`yield()` Hàm này là lớp bọc đơn giản nhất để một process tự nguyện nhường CPU, thường gọi mỗi khi có ngắt timer:

- `acquire(&p->lock)` để chuẩn bị đổi `state` an toàn.
- Đặt `p->state = RUNNABLE`, báo cho scheduler biết process này vẫn còn việc để chạy tiếp, chỉ là đang tạm nhường lượt.
- Gọi `sched()` để thực hiện việc chuyển ngữ cảnh về `scheduler()`.
- Khi scheduler chọn process này chạy lại ở một lượt sau, luồng thực thi quay lại đúng ngay sau lời gọi `sched()`, và `release(&p->lock)` được gọi để kết thúc `yield()`.

2 Phần cài đặt: system call `sysinfo`

2.1 Mô tả yêu cầu

Mục tiêu tổng quát

Yêu cầu của bài lab là xây dựng một system call mới tên `sysinfo` cho hệ điều hành xv6, cho phép chương trình user thu thập thông tin tổng quan về trạng thái tài nguyên của toàn bộ hệ thống tại thời điểm gọi. System call này giúp người dùng quan sát được hai nhóm tài nguyên cốt lõi mà kernel đang quản lý: các file đang mở và các tiến trình đang hoạt động.

Việc cài đặt `sysinfo` đòi hỏi phải hiểu rõ cách kernel xv6 tổ chức bảng file toàn cục `ftable`, bảng file descriptor cục bộ `ofile[]` của từng tiến trình, cũng như mảng tiến trình `proc[]` cùng cơ chế lập lịch. Ngoài ra, vì dữ liệu được thu thập từ bên trong kernel nhưng phải gửi về cho chương trình user, nên cần xử lý đúng cơ chế sao chép dữ liệu qua ranh giới user-kernel.

Đặc tả thông tin về file đang mở

System call `sysinfo` cần thu thập toàn bộ các file đang được mở trong hệ thống, tức là các ô trong mảng `ftable.file[]` có trường `ref > 0`. Với mỗi file đang mở, cần ghi nhận các thông tin sau:

- Loại file: phân biệt giữa file thường trên đĩa `FD_INODE`, cơ chế Pipe trong giao tiếp giữa các tiến trình `FD_PIPE`, hay thiết bị ngoại vi `FD_DEVICE`.
- Chế độ truy cập: file đang được mở để đọc, ghi, hay cả hai, dựa trên hai trường `readable` và `writable` của `struct file`.
- Số lượng tham chiếu `ref`: cho biết có bao nhiêu file descriptor trên toàn hệ thống đang trỏ tới `struct file` này. Giá trị này lớn hơn 1 khi file được chia sẻ thông qua `dup()` hoặc `fork()`.
- Danh sách các tiến trình đang tham chiếu: liệt kê PID của tất cả các tiến trình có ít nhất một phần tử trong `ofile[]` trỏ tới `struct file` đang xét.

Đặc tả thông tin về tiến trình

Bên cạnh thông tin file, `sysinfo` cần thu thập thông tin về tất cả các tiến trình đang hoạt động, tức là các ô trong mảng `proc[]` có trạng thái khác `UNUSED`. Với mỗi tiến trình, cần ghi nhận:

- Định danh tiến trình `pid`.
- Trạng thái hiện tại `state`: một trong các giá trị `USED`, `SLEEPING`, `RUNNABLE`, `RUNNING`, `ZOMBIE`.

- Định danh tiến trình cha `ppid`: lấy từ `p->parent->pid`. Với tiến trình `init` không có cha, giá trị này bằng 0.
- Số lần được CPU lập lịch thực thi: đếm mỗi lần bộ lập lịch `scheduler()` chọn tiến trình này và chuyển trạng thái sang `RUNNING`.
- Danh sách các file descriptor đang mở: liệt kê chỉ số `fd` mà ô `ofile[fd]` tương ứng khác rỗng.

Giao diện lời gọi hệ thống

Từ phía user, system call có prototype là `int sysinfo(struct sysinfo *info)`, trong đó `info` là con trỏ trỏ tới một vùng nhớ thuộc không gian địa chỉ của tiến trình gọi. Kernel sẽ thu thập dữ liệu rồi sao chép toàn bộ cấu trúc `struct sysinfo` vào vùng nhớ mà con trỏ này chỉ tới.

Giá trị trả về bằng 0 nếu thành công. Nếu con trỏ `info` không hợp lệ, ví dụ bằng `NULL` hoặc nằm ngoài phạm vi bộ nhớ của tiến trình, kernel phải trả về -1 mà không được gây panic hay làm sập hệ thống.

2.2 Thiết kế

Thiết kế cấu trúc dữ liệu

Để chứa toàn bộ thông tin thu thập được, ta định nghĩa ba cấu trúc dữ liệu mới trong file `kernel/sysinfo.h`.

Cấu trúc `struct file_info` lưu thông tin của một file đang mở, gồm các trường `type` cho biết loại file, `readable` và `writable` cho biết chế độ truy cập, `ref` là số lượng tham chiếu, `num_pids` là số tiến trình đang tham chiếu, và mảng `pids[NPROC]` chứa danh sách PID tương ứng.

Cấu trúc `struct proc_info` lưu thông tin của một tiến trình, gồm `pid`, `state`, `ppid`, `sched_count` là số lần được lập lịch, `num_open_files` là số file đang mở, và mảng `open_files[NOFILE]` chứa danh sách các fd hợp lệ.

Cấu trúc bao ngoài `struct sysinfo` gom hai mảng: `files[NFILE]` chứa tối đa 100 bản ghi file và `procs[NPROC]` chứa tối đa 64 bản ghi tiến trình, cùng hai biến đếm `num_files` và `num_procs`. Kích thước các mảng được chọn bằng đúng các hằng số cấu hình của xv6 tại `kernel/param.h`, đảm bảo không bao giờ bị tràn vì hệ thống không thể có nhiều file hay tiến trình hơn giới hạn tĩnh này.

Thiết kế cơ chế đếm số lần lập lịch

Để theo dõi số lần mỗi tiến trình được CPU lập lịch thực thi, ta thêm một trường `int sched_count` vào `struct proc` tại `kernel/proc.h`. Trường này nằm cạnh các trường được bảo vệ bởi `p->lock` như `state` và `pid`.

Giá trị `sched_count` được gán bằng 0 tại hai vị trí trong `kernel/proc.c`: trong hàm `allocproc()` khi một ô trống trong mảng `proc[]` vừa được cấp phát cho tiến trình mới, và trong hàm `freeproc()` khi tiến trình bị thu hồi để trả ô về trạng thái `UNUSED`.

Điểm đếm nằm trong hàm `scheduler()`. Mỗi khi bộ lập lịch duyệt mảng `proc[]` và tìm thấy một tiến trình có trạng thái `RUNNABLE`, nó chuyển tiến trình đó sang `RUNNING` rồi gọi `swtch()` để cấp CPU. Ngay trước lệnh `swtch()`, ta chèn thêm dòng `p->sched_count++` để ghi nhận sự kiện lập lịch này. Vì lúc đó `p->lock` đang được giữ nên thao tác tăng biến là an toàn.

Thuật toán thu thập dữ liệu và chiến lược đồng bộ

Khi thu thập thông tin cho `sysinfo`, điểm cần chú ý nhất là đồng bộ hóa vì ta phải đọc dữ liệu từ cả bảng tiến trình `proc[]` lẫn bảng file `ftable.file[]`. Mỗi bảng lại có cơ chế khóa riêng là `p->lock` và `ftable.lock`. Nếu một CPU vừa giữ `ftable.lock` vừa chờ xin `p->lock`,

trong khi CPU khác đang giữ `p->lock` lại chờ `ftable.lock`, hệ thống sẽ rơi vào deadlock và đứng máy.

Cách giải quyết là chia việc thu thập thành hai bước riêng biệt, không bao giờ giữ đồng thời cả hai loại khóa.

Ở bước đầu tiên, ta duyệt mảng `proc[]`. Do trường `parent` được bảo vệ bởi `wait_lock`, nên để tránh deadlock với `wait()` và `exit()`, ta cần lấy `wait_lock` trước rồi mới lấy `p->lock`. Đọc xong các thông tin như `pid`, `state`, `sched_count`, `ppid` và sao chép tạm mảng con trỏ `ofile[]`, ta nhả `p->lock` rồi nhả `wait_lock`. Khi quét xong toàn bộ tiến trình, dữ liệu đã nằm gọn trong mảng tạm và kernel không còn giữ khóa nào.

Sang bước thứ hai, ta giữ `ftable.lock` rồi duyệt mảng `ftable.file[]`. Ô nào có `ref > 0` thì trích xuất `type`, `readable`, `writable`, `ref`, sau đó đối chiếu với mảng con trỏ đã lưu ở bước một để tìm các PID đang mở file này. Bước này chỉ dùng mỗi `ftable.lock` nên hoàn toàn không có nguy cơ deadlock.

Thiết kế quản lý bộ nhớ và truyền dữ liệu User-Kernel

Cấu trúc `struct sysinfo` có kích thước khá lớn cỡ vài chục KB do chứa các mảng tĩnh lưu thông tin file và tiến trình. Trong khi đó, vùng nhớ tạm kernel stack của mỗi tiến trình trong xv6 rất hạn chế, chỉ rộng đúng 4096 byte. Nếu khai báo `struct sysinfo` làm một biến cục bộ trên stack, hệ thống chắc chắn sẽ bị tràn bộ nhớ và dẫn đến sập kernel.

Để giải quyết, ta sử dụng một biến tĩnh toàn cục `static struct sysinfo k_sysinfo` trong `kernel/sysproc.c`. Biến này được lưu cố định ở vùng nhớ dùng chung của kernel nên hoàn toàn không tiêu tốn không gian stack. Do biến này dùng chung cho toàn hệ thống, ta bảo vệ nó bằng một khóa `sysinfo_lock`, đảm bảo tại mỗi thời điểm chỉ có duy nhất một tiến trình được phép ghi dữ liệu.

Sau khi thu thập xong dữ liệu, hàm `sys_sysinfo()` sử dụng hàm `copyout()` để sao chép an toàn dữ liệu từ kernel sang vùng nhớ phía user. Nếu địa chỉ do tiến trình user truyền vào không hợp lệ, `copyout()` sẽ báo lỗi để kernel trả về -1 cho user, giúp hệ thống xử lý an toàn mà không bị crash.

2.3 Cài đặt

Danh sách file thay đổi

File	Nội dung thay đổi
`kernel/sysinfo.h`	Định nghĩa 3 cấu trúc dữ liệu: `struct file_info`, `struct proc_info`, `struct sysinfo`.
`kernel/syscall.h, syscall.c`	Định nghĩa số hiệu `SYS_sysinfo` (23), khai báo nguyên mẫu và ánh xạ vào mảng `syscalls[]`.
`kernel/sysproc.c`	Khai báo biến tĩnh toàn cục `k_sysinfo`, khóa `sysinfo_lock`, bảng chụp tạm `ofile_snap`, `pid_snap`; cài đặt hàm điều phối `sys_sysinfo()`.
`kernel/proc.h, proc.c`	Thêm `sched_count` vào `struct proc`; reset về 0 trong `allocproc()`, `freeproc()`; tăng biến đếm trong `scheduler()`.
`kernel/file.c, defs.h`	Hiện thực hàm `get_file_info()` duyệt `ftable.file[]` dưới sự bảo vệ của `ftable.lock`; khai báo nguyên mẫu trong `kernel/defs.h`.
`user/user.h, usys.pl`	Khai báo prototype cho user và thêm entry stub hợp ngữ.
`user/sysinfotest.c, Makefile`	Hiện thực chương trình kiểm thử 8 kịch bản và đăng ký biên dịch vào biến `UPROGS`.

Bảng 3: Các file được thêm/sửa

***Tổng quan luồng xử lý của* `sys_sysinfo()`**

Hàm thực thi tuần tự theo 5 bước:

1. Lấy địa chỉ đích `addr` từ thanh ghi đối số phía user thông qua `argaddr(0, &addr)`.
2. Nắm giữ `sysinfo_lock`, dùng `memset()` xóa sạch dữ liệu cũ trong `k_sysinfo`.
3. **Pha 1:** Quét `proc[]`, trích xuất dữ liệu tiến trình và lưu bản chụp `ofile_snap`, `pid_snap`.
4. **Pha 2:** Gọi `get_file_info()` quét `ftable.file[]` và đối chiếu với `ofile_snap`.
5. Sao chép `k_sysinfo` về không gian user bằng `copyout()`, nhả `sysinfo_lock` và trả về mã trạng thái (0 hoặc -1).

Thứ tự khóa: `sysinfo_lock` → `wait_lock` → `p->lock` ở Pha 1 và `sysinfo_lock` → `ftable.lock` ở Pha 2, triệt tiêu hoàn toàn khả năng hình thành chu trình chờ khóa.

Thu thập thông tin tiến trình (Pha 1)

Thực hiện trong `sys_sysinfo()`:

- Duyệt i từ 0 đến `NPROC - 1`, lần lượt gọi `acquire(&wait_lock)` rồi `acquire(&p->lock)`.
- Bỏ qua nếu `p->state == UNUSED`.
- Với tiến trình hợp lệ: lưu `pid`, `state`, `sched_count`; xác định `ppid` qua `p->parent` (0 nếu không có cha).
- Quét `p->ofile[]`: ghi nhận các chỉ số fd hợp lệ vào `open_files[]`, đồng thời sao chép con trỏ file vào `ofile_snap[idx][fd]` và lưu `pid_snap[idx] = p->pid`.
- Nhả `p->lock`, nhả `wait_lock` và tăng `k_sysinfo.num_procs`.

Thu thập thông tin file đang mở (Pha 2)

Hàm `get_file_info()` (`kernel/file.c`) giữ khóa `ftable.lock` và duyệt mảng `ftable.file[]`:

- Bỏ qua các file có `ref == 0`.
- Với file đang mở: trích xuất `type`, `readable`, `writable`, `ref`.
- Xác định các PID đang giữ file bằng cách duyệt bảng `ofile_snap`: nếu `ofile_snap[p][fd] == f`, thêm `pid_snap[p]` vào mảng `pids[]` và ngắt duyệt fd để mỗi PID chỉ ghi nhận duy nhất một lần (xử lý đúng trường hợp `dup()`).
- Kết thúc duyệt thì nhả `ftable.lock`.

Sao chép kết quả về user space

Gọi `copyout(curproc->pagetable, addr, (char *)&k_sysinfo, sizeof(k_sysinfo))`. Hàm kiểm tra ánh xạ bảng trang và các cờ phân trang (`PTE_U`, `PTE_W`):

- Nếu địa chỉ hợp lệ: hoàn tất việc ghi dữ liệu, nhả `sysinfo_lock`, trả về 0.
- Nếu địa chỉ sai (con trỏ rác, vùng text chỉ đọc, hoặc ngoài giới hạn): `copyout()` trả về lỗi; kernel nhả `sysinfo_lock` và trả về -1 an toàn.

2.4 Chương trình kiểm thử `user/sysinfotest.c`

Các trường hợp kiểm thử

STT	Kịch bản kiểm thử	Kết quả mong đợi	Đánh giá
1	Trạng thái ban đầu sau khởi động	Trả về 0; `num_files = 1` (console `FD_DEVICE`, `ref = 9`, `pids`={1,2,3}); `num_procs = 3` (`init`, `sh`, `sysinfotest`); `init.ppid = 0`.	PASS
2	Mở file (`open("README", O_RDONLY)`)	`num_files` tăng 1; file mới loại `FD_INODE`, $r = 1, w = 0, ref = 1$; tiến trình có thêm fd 3.	PASS

STT	Kịch bản kiểm thử (tiếp)	Kết quả mong đợi	Đánh giá
3	Đóng file (`close(fd)`)	`num_files` giảm 1 về mức ban đầu; fd 3 biến mất khỏi danh sách `open_files`.	PASS
4	Nhân bản fd qua `dup(fd)`	`num_files` không tăng thêm; file có `ref = 2`, `num_pids = 1`; tiến trình sở hữu cả fd 3 và fd 4.	PASS
5	Chia sẻ file qua `fork()`	File có `ref = 2`, `num_pids = 2` (gồm cả cha và con); con có `ppid == parent.pid`.	PASS
6	Tạo đường ống qua `pipe(p)`	`num_files` tăng 2 (`FD_PIPE`, $ref = 1$): đầu đọc ($r = 1, w = 0$) và đầu ghi ($r = 0, w = 1$).	PASS
7	Kiểm tra tiến trình con `ZOMBIE`	`num_procs` tăng 1; tiến trình con có `state = ZOMBIE`, `sched_count > 0`, `num_open_files = 0`.	PASS
8	Truyền con trỏ lỗi (`NULL`, ngoài dải địa chỉ)	Trả về -1; kernel không panic hay bị crash.	PASS

Bảng 4: Các kịch bản kiểm thử `sysinfo`

Kết quả thực thi

Đầu ra console khi chạy `sysinfotest` trên xv6:

```
$ sysinfotest
=== Starting sysinfotest ===
[PASS] Test 1 (Initial state): num_files=1 (FD_DEVICE, ref=9,
pids=1,2,3), num_procs=3, init ppid=0
[PASS] Test 2 (Open file): num_files=2, FD_INODE r=1 w=0 ref=1,
new fd=3
[PASS] Test 3 (Close file): num_files=1, fd 3 gone
[PASS] Test 4 (dup fd): num_files=2, ref=2, num_pids=1, fds=3,4
[PASS] Test 5 (fork shared file): ref=2, num_pids=2, child
ppid=3 == parent pid
[PASS] Test 6 (pipe): num_files increased by 2, FD_PIPE ref=1
(read & write ends)
[PASS] Test 7 (ZOMBIE process): child PID=5, state=ZOMBIE,
sched_count>0, open_files=0
[PASS] Test 8 (Invalid pointer): NULL -> -1, 0xffffffffffffffff
-> -1, no panic
=== ALL TESTS PASSED! ===
$
```

3 Kết luận

3.1 Kết quả đạt được

Qua bài lab, nhóm đã tìm hiểu chi tiết cách xv6 tổ chức quản lý file và tiến trình, đồng thời hiện thực thành công system call `sysinfo`. Từ phía user space, chương trình kiểm thử có thể thu thập đầy đủ thông tin hệ thống tại thời điểm gọi, bao gồm số lượng và danh sách các file đang mở cũng như danh sách các tiến trình đang hoạt động.

Bộ kiểm thử `sysinfotest` đã chạy thành công các kịch bản đặt ra, từ việc kiểm tra trạng thái mặc định khi vừa khởi động, các thao tác mở, đóng, `dup`, `pipe`, kiểm tra tiến trình con chuyển sang trạng thái `ZOMBIE`, cho đến việc xử lý an toàn khi truyền con trỏ lỗi mà không làm sập hệ thống.

3.2 Kiến thức và bài học rút ra

Quá trình làm bài lab giúp nhóm làm rõ nhiều khái niệm thực tế trong kernel xv6:

- Cơ chế quản lý file hai tầng: Bảng `ofile[]` ở mỗi tiến trình chỉ lưu con trỏ trỏ về mảng toàn cục `ftable.file[]`. Điều này giải thích tại sao khi hệ thống vừa khởi động, cả 3 tiến trình `init`, `sh` và `sysinfotest` đều có các fd 0, 1, 2 nhưng thực chất cả 9 fd này đều trỏ chung vào đúng một struct file của thiết bị console, khiến biến đếm `ref` của console bằng 9.
- Cơ chế lập lịch round-robin: Hiểu rõ cách hàm `scheduler()` trên mỗi CPU liên tục quét mảng `proc[]` để tìm tiến trình `RUNNABLE` rồi gọi `swtch()` chuyển quyền thực thi. Biến đếm `sched_count` thêm vào đã phản ánh chính xác số lần mỗi tiến trình được CPU chọn chạy.
- Thứ tự khóa và tránh deadlock: Khi cần lấy dữ liệu từ nhiều cấu trúc khác nhau, việc quản lý khóa rất dễ gây lỗi nếu không cẩn thận. Nhóm đã chia việc thu thập làm hai pha riêng để không bao giờ giữ cùng lúc `p->lock` và `ftable.lock`. Đặc biệt, khi đọc trường `parent`, việc lấy `wait_lock` trước `p->lock` là bắt buộc để không bị deadlock với các hàm `wait()` và `exit()`.
- Giới hạn bộ nhớ stack trong kernel: Do kernel stack của mỗi tiến trình trong xv6 chỉ có đúng một trang 4096 byte, việc khai báo một struct lớn như `sysinfo` làm biến cục bộ sẽ gây tràn stack ngay lập tức. Giải pháp dùng biến tĩnh `static` kết hợp với `spinlock` là kinh nghiệm thực tế rất quan trọng khi lập trình trong không gian kernel.

3.3 Đánh giá mức độ hoàn thành

Nhóm đã hoàn thành toàn bộ các yêu cầu của bài lab: phân tích lý thuyết, hiện thực mã nguồn system call và viết chương trình kiểm thử bao quát các trường hợp. Hệ thống biên dịch và chạy ổn định trên QEMU, các kết quả thu được đều khớp với phân tích lý thuyết.